UNIVERSITY OF GHANA

SCHOOL OF ENGINEERING SCIENCES
DEPARTMENT OF COMPUTER ENGINEEERING
2ND SEMESTER,2025/2026 ACADEMIC YEAR
COURSE: CPEN 208-SOFTWARE ENGINEERING
LECTURER: MR JOHN ASIAMAH
PROJECT 1

Table of Contents

1. Introduction

This project has two parts, as required by the assignment. The first is a PostgreSQL database for the Computer Engineering Department, covering student information, fee payments, course enrollment, and staff assignments (lecturer-to-course and lecturer-to-TA). The second is a Next.js 14 web app with login, registration, and a student dashboard, connected directly to that database.

The database is called CPEN208_PROJECT1. Everything sits in PostgreSQL's default public schema, since the whole system is one connected domain and did not need to be split into separate schemas.

Each of the five required functionalities is covered as follows:

- Student Personal Information — the student table, shown on the dashboard and used during login/registration.
- Student Fees Payments — the student_fees table, plus a function (get_outstanding_fees) that returns each student's balance as JSON.
- Course Enrollment — the enrollment table, which students can actually add to or remove from on the dashboard.
- Lecturer to Course Assignment — the lecturer_course table, filled with the department's real Level 200 courses and lecturers.
- Lecturer to TA Assignment — the lecturer_ta table, filled with the department's real teaching assistants.

The last two are handled at the database level with correct, real sample data. The assignment only asks for login, register, and dashboard pages in the web app, not an admin screen for managing staff assignments, so no extra page was built for that.

2. Database Design

2.1 Assumptions

A few decisions had to be made that weren't spelled out in the assignment. Here they are, and why:

- Database name: CPEN208_PROJECT1, in the default public schema.
- A Teaching Assistant (TA) is someone who has already graduated, not a current student. So a TA has its own table, separate from student, with no fees or enrollment.
- Only students log into the app. Lecturers and TAs don't need accounts, so their tables only store a name — nothing else.
- The department fee is billed once a year, not once a semester. So student_fees has one row per student per academic year, and the outstanding balance is just amount_billed minus amount_paid.
- Courses and staff assignments are tied to a specific semester, since those do change from one semester to the next, even though the fee itself is yearly.
- If a new student registers who wasn't already in the seeded data, they're automatically billed the standard fee right away, so their balance is correct from the start.

2.2 Tables and How They Relate

There are 8 tables in total: student, lecturer, ta, and course stand on their own, student_fees belongs to a student, and enrollment, lecturer_course, and lecturer_ta connect two tables together at once.

Relationship	Type	Explanation
student → student_fees	One-to-many	A student can have more than one fee record; the balance is added up across all of them.
student ↔ course	Many-to-many	Handled by the enrollment table — a student takes many courses, a course has many students.
lecturer ↔ course	Many-to-many	Handled by the lecturer_course table, per semester.
lecturer ↔ ta	Many-to-many	Handled by the lecturer_ta table, tied to a specific course too.

2.3 Table Structure

student

Column	Type	Notes
student_id	VARCHAR(15)	Primary key
first_name / last_name	VARCHAR(100)	Required
email	VARCHAR(150)	Unique, filled in only after the student registers
password_hash	VARCHAR(255)	Filled in only after the student registers
program	VARCHAR(100)	Defaults to 'Computer Engineering'
level	INT	Defaults to 200
phone	VARCHAR(20)	Optional
date_registered	TIMESTAMP	Set when the student creates/claims their account

lecturer

Column	Type	Notes
lecturer_id	VARCHAR(15)	Primary key
first_name / last_name	VARCHAR(100)	Required

ta

Column	Type	Notes
ta_id	VARCHAR(15)	Primary key
first_name / last_name	VARCHAR(100)	Required

course

Column	Type	Notes
course_id	VARCHAR(15)	Primary key, e.g. 'CPEN204'
course_title	VARCHAR(150)	Required

credit_hours	INT	Required
level	INT	Defaults to 200

student_fees

Column	Type	Notes
fee_id	SERIAL	Primary key
student_id	VARCHAR(15)	Linked to student
semester	VARCHAR(30)	Academic year, e.g. '2025/2026'
amount_billed	NUMERIC(10,2)	Required
amount_paid	NUMERIC(10,2)	Defaults to 0

enrollment

Column	Type	Notes
enrollment_id	SERIAL	Primary key
student_id	VARCHAR(15)	Linked to student
course_id	VARCHAR(15)	Linked to course
semester	VARCHAR(30)	Required
date_enrolled	DATE	Defaults to today

lecturer_course

Column	Type	Notes
id	SERIAL	Primary key
lecturer_id	VARCHAR(15)	Linked to lecturer
course_id	VARCHAR(15)	Linked to course
semester	VARCHAR(30)	Required

lecturer_ta

Column	Type	Notes
id	SERIAL	Primary key
lecturer_id	VARCHAR(15)	Linked to lecturer
ta_id	VARCHAR(15)	Linked to ta
course_id	VARCHAR(15)	Linked to course
semester	VARCHAR(30)	Required

2.4 Sample Data

The database is filled with real data from the actual class, not placeholders:

- 69 real Level 200 students, plus 2 demo accounts that can log in right away for testing.
- 9 real courses: CPEN201, CPEN203, CPEN204, CPEN206, CPEN208, CPEN211, CPEN212, CPEN213, CPEN214.

- 9 real lecturers and 4 real teaching assistants, correctly matched to their courses across both semesters.
- Every student is enrolled in every course for their level, and every student has a fee record with a realistic, varied amount already paid.

3. Outstanding Fees Function

The assignment asks for a database function that works out each student's outstanding fees and returns it as a JSON array. Here it is:

```
CREATE OR REPLACE FUNCTION get_outstanding_fees()
RETURNS JSON AS $$
DECLARE
    result JSON;
BEGIN
    SELECT
json_agg(         json_build_object(
            'student_id', sub.student_id,
            'first_name', sub.first_name,
            'last_name', sub.last_name,
            'total_billed', sub.total_billed,
            'total_paid', sub.total_paid,
            'outstanding_balance', sub.outstanding_balance
        )
    ) INTO result
    FROM (
        SELECT s.student_id, s.first_name, s.last_name,
            COALESCE(SUM(f.amount_billed), 0) AS total_billed,
            COALESCE(SUM(f.amount_paid), 0) AS total_paid,
            COALESCE(SUM(f.amount_billed) - SUM(f.amount_paid), 0) AS outstanding_balance
FROM student s
        LEFT JOIN student_fees f ON s.student_id = f.student_id
        GROUP BY s.student_id, s.first_name, s.last_name
    ) sub;
    RETURN result;
END;
$$ LANGUAGE plpgsql;
```

It works out each student's total billed and total paid first, then turns each student into a small JSON object, and combines all of them into one JSON array. A LEFT JOIN makes sure a student with no fee record still shows up with zero values instead of disappearing. It's called with SELECT get_outstanding_fees();, and the dashboard uses this same function to show each student their own balance.

4. The Web Application

Built with Next.js 14 (App Router) and Tailwind CSS, connected to PostgreSQL with the pg library. Passwords are hashed with bcrypt before being saved, and every query uses parameters so the app is not open to SQL injection.

4.1 Registration

The registration form asks for a student ID, name (only needed for new students), email, phone, and password. It handles three cases:

- The student ID is already fully registered — the form rejects it and tells the person to log in instead.

- The student ID exists but hasn't been claimed yet (one of the 69 seeded students) — their record is updated with the details just entered.
- The student ID doesn't exist at all — a new student record is created from scratch.

A new student is also billed the standard fee automatically, so their balance is correct straight away.

4.2 Login

The login page checks a student's email and password against the database. The password is hashed and compared to the saved hash; if it matches, the student's details (never the password itself) are sent back and used to load the dashboard.

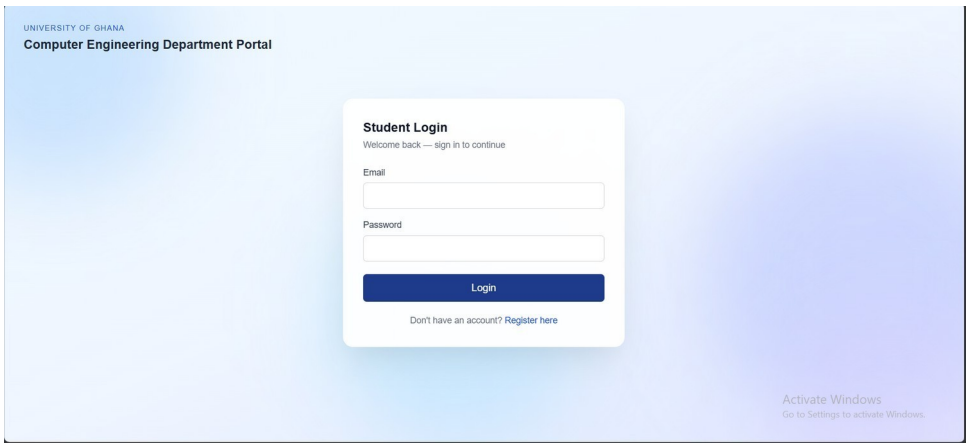

Figure 1 — Student login page

4.3 Dashboard

After logging in, a student sees their personal details, their fee summary (billed, paid, and outstanding, straight from get_outstanding_fees()), and their courses — split into what they're enrolled in and what's still available. Each course shows its lecturer and TA. Students can enroll in or drop a course directly from this page, and it updates right away.

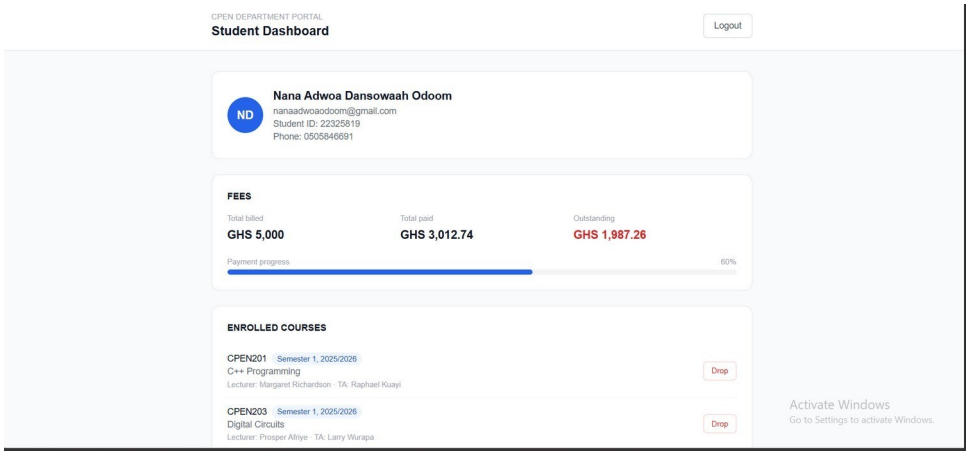

Figure 2 — Student dashboard

5. What Was Submitted

Everything was pushed to a public GitHub repository, containing:

- The full Next.js 14 source code.

- All database scripts, in a database/ folder: create_tables.sql, functions.sql, and insert_data.sql.
- A full database backup (cpen208_backup.sql) made with pgAdmin.

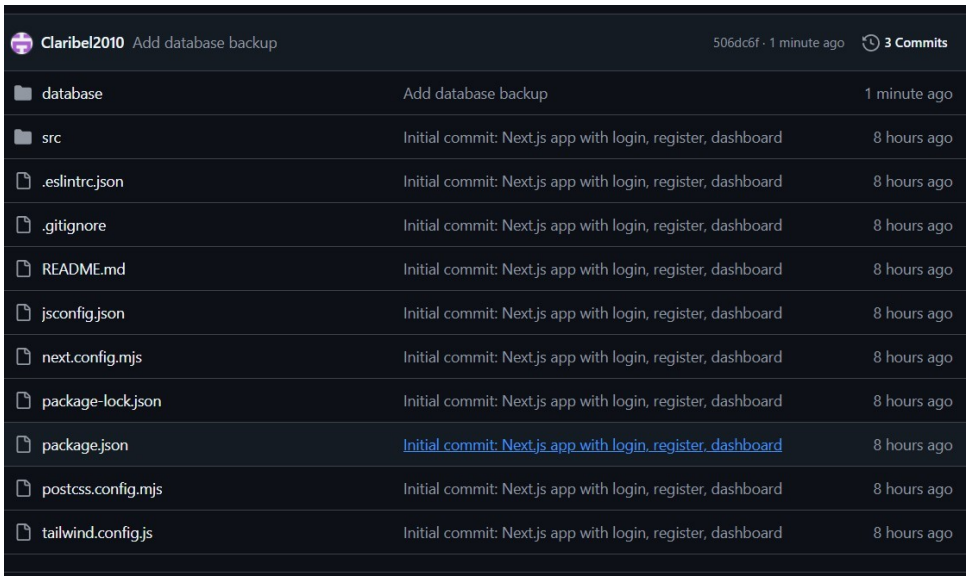

Figure 3 — GitHub repository

Repository: https://github.com/Claribel2010/cpen208_project1

6. Conclusion

This project covers all five required functionalities: student information, fee payments (with the required outstanding-fees function), course enrollment, lecturer-to-course assignment, and lecturerto-TA assignment. It uses real data from the actual Level 200 class rather than made-up examples, and the web app lets a student register, log in, and interact with a live dashboard connected to PostgreSQL.

The source code, database scripts, database backup, and this report are all available in the GitHub repository linked above.